LA ROCHELLE UNIVERSITÉ

LICENSE 3 INFORMATIQUE

Modélisation de bases de données

Project Report:
Designing Postgres Database for Genealogy dataset

Professor(s) BOUWMANS Thierry

Members TRAN Minh Duong

PHAM Ngan Ha

NGUYEN Hoang Nhat

Abstract

This project builds a database to organize and study marriage records from France. We take 5,000 marriage documents and create a PostgreSQL database to store them in an organized way. The goal is to design the database so that it is well-organized, easy to use, and can answer questions about families and dates. This report explains how we organized the data and built the database system.

I. Introduction

Old documents and records from France contain important information about families and history. Many of these documents are stored in archives. To study and understand this information, we need to organize it in a database. This project focuses on creating a database system for marriage records from the Vendée region. We show how to take messy information and organize it in a clean, useful way.

I.1. Dataset Overview

We use a file called `mariages_L3_5k.csv` which contains 5,000 marriage records. Each record includes information about two people who got married, their parents, and the place and date of marriage.

The data has these main parts:

- **Record Information:** Each record has an ID number and a date
- **Person A Information:** Name of the first person, and names of their mother and father
- **Person B Information:** Name of the second person, and names of their mother and father
- **Location Information:** The city (commune) and region (department) where the marriage happened

I.2. Why This Project Is Important

When we have many old documents, it is difficult to find information or see patterns. A database makes this easier. Our database has these goals:

1. **Organization:** Store all the information in one place in a clear way
2. **No Duplicates:** Each person should appear only once in the database, even if they appear in many records
3. **Correct Information:** Make sure that all relationships between people and places are correct

II. Database Design & Normalization

II.1. Initial Conceptualization

Initially, we treat the dataset as a single flat table, where the Act ID is the primary key and every other attribute depends on it

Corresponding FD:

```
Act ID → (
    Act Type,
    Person A's Last Name,
    Person A's First Name,
    First Name of A's Father,
    Last Name of A's Mother,
    First Name of A's Mother,
    Person B's Last Name,
    Person B's First Name,
    First Name of B's Father,
    Last Name of B's Mother,
    First Name of B's Mother,
    Town,
    Department,
    Date,
    View Number
)
```

=> Result: 1NF achieved

II.2. Normalization Process

1. **For 1NF:**
   All values are atomic (each cell contains one piece of data).

   Each record is uniquely identified by an Act ID => Result: 1NF achieved

2. **For 2NF:**
   Problem: Repetition of data for parents, towns, and departments leads to redundancy.

   Solution: Separate entities so that every non-key attribute depends on the whole primary key

   Resulting FDs:

   ```
   - Act_ID → (Type, Date, View_Num, Commune_ID, Person_A_ID, Person_B_ID)
   - Person_ID → (Last_Name, First_Name, Father_ID, Mother_ID)
   - Commune_ID → (Name, Dept_Code)
   ```

   => Result: 2NF achieved

3. **For 3NF:**
   Problem:

   - Transitive Dependency: The Department depended on the Town, which depended on the Act.
   - Parental names depended on the person

Solution:

- Extracted Department into a standalone table to enforce valid codes (44, 49, 79, 85).
- Established foreign keys `(father_id, mother_id)` within the Person table to handle lineage without repeating names.

Final FDs:

```
Act_ID → (Act_Type, Date, View_Num, Commune_ID, Person_A_ID, Person_B_ID)
Person_ID → (Last_Name, First_Name, Father_ID, Mother_ID)
Commune_ID → (Commune_Name, Dept_Code)
Dept_Code → (Valid_Codes: 44, 49, 79, 85)
```

=> Result: 3NF achieved

II.3. Final Relation Schema

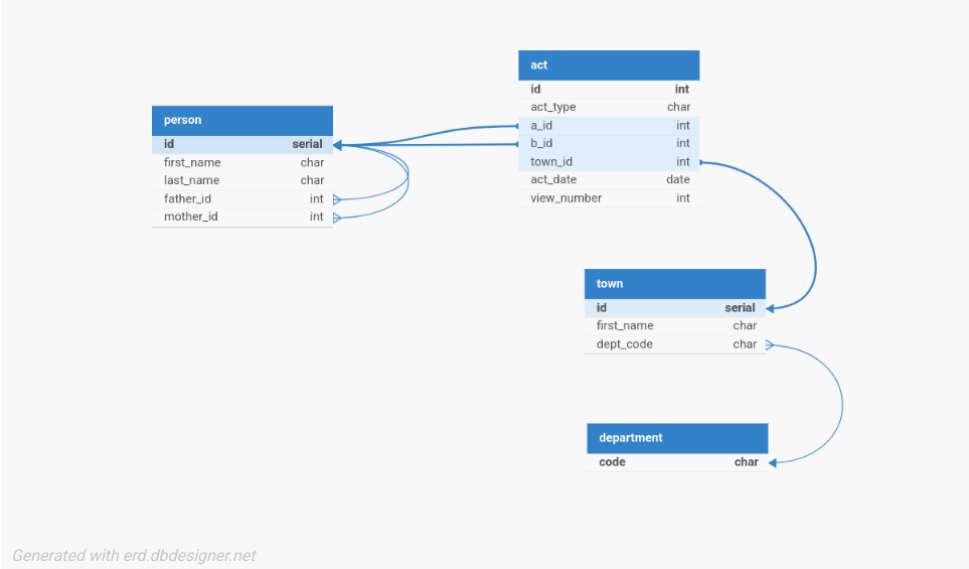


Figure 1: Database Relation Schema

Cardinality:

- Each parent can have more than one child but a child can only have one of each parent.
- Each department can have more than one town, but a town must only belong to one department.

III. Data Extraction & ETL Process

To get our data from the messy CSV file into our clean database, we created a two-step process using Python scripts. The first script, `utils/data_parser.py`, handles the cleaning and organizing, and the second script, `main.py`, loads it into the database.

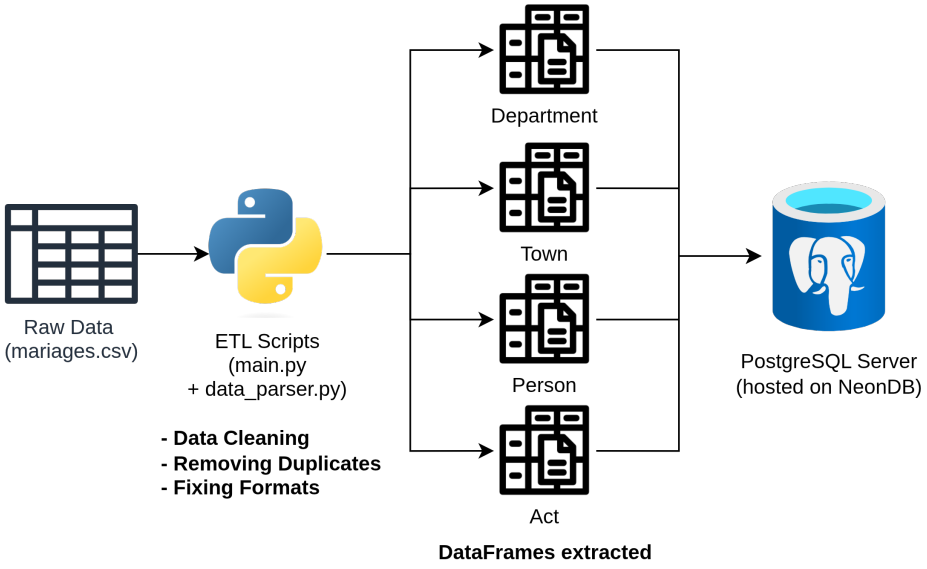


Figure 2: Overview of the ETL Process

III.1. Extraction

First, we pull all the data from the `mariages_L3_5k.csv` file. We used the pandas library in Python to help with this. An important step was to read everything as text first, so we didn't accidentally change data, like dropping the '0' from a department code like '05'. We also made sure to use UTF-8 encoding, which helps read special French characters correctly.

III.2. Transformation

This is where we do most of the cleanup work. Our script goes through the raw data row by row and gets it ready for our normalized tables.

Data Cleaning

We made a simple cleaning function that does two things:

- It removes any extra spaces from the beginning or end of each piece of data.
- It finds any empty or "n/a" values and marks them as properly missing, so we can handle them consistently.

Finding and Removing Duplicates

To avoid storing the same person or town multiple times, our script keeps track of what it has already seen.

- **Departments and Towns:** It checks for valid department codes and skips any rows with bad data. It makes sure a town is unique by looking at both its name and its department code.
- **People:** To decide if a person is unique, we look at their full name and also who their parents are. This helps tell apart two people who might have the same name.

Fixing Formats

We also had to fix a few things to match our database design:

- **Dates:** We converted the date from the "DD/MM/YYYY" text format into a real date that the database can understand.
- **View Number:** We took columns like "48/210" and just kept the first number (48), which is the page number we care about.

III.3. Loading

Instead of manually importing the files, we used a second Python script, `main.py`, to automatically load our cleaned data into the PostgreSQL database. This script uses a library called SQLAlchemy, which helps Python talk to databases.

The loading process is done carefully to avoid errors:

1. **Final Check:** Before loading, the script does one last check on our cleaned data tables to make sure all the ID numbers are valid integers and that no essential information is missing.
2. **Order of Operations:** It loads the tables in the correct order: first `department`, then `town` (which needs departments to exist), then `person`, and finally `act` (which needs towns and people).
3. **Handling People:** The `person` table was a bit tricky because a person's parents are also in the same table. To solve this, we first insert all people with their parent fields empty. Then, once everyone is in the table, we go back and add the parent-child links.
4. **Batching:** To make it faster, the script inserts the data in large chunks (batches) instead of one row at a time.
5. **Updating Sequences:** After inserting everything with specific IDs, we reset the database's automatic ID counters. This ensures that if we add new data later, we won't have any ID conflicts.

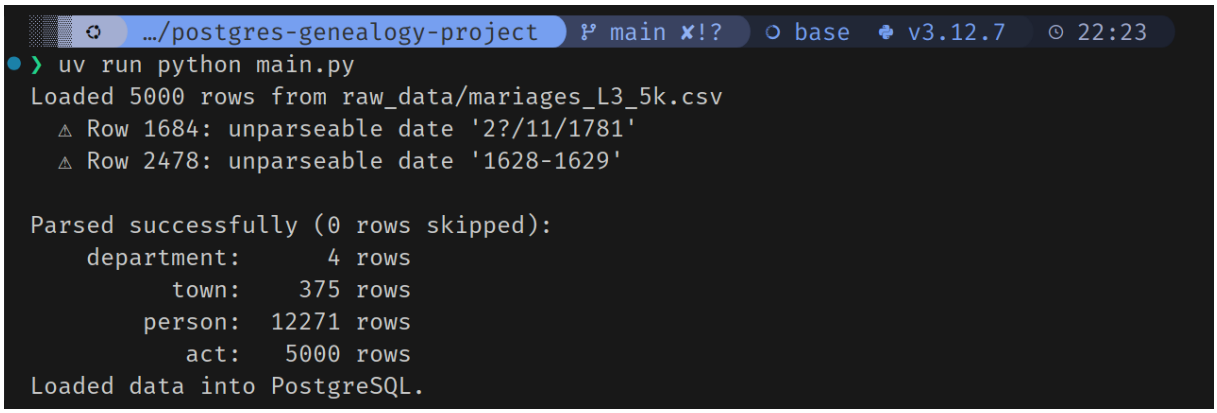


Figure 3: Terminal output after running the ETL pipeline with main.py

IV. Genealogical Queries & Results

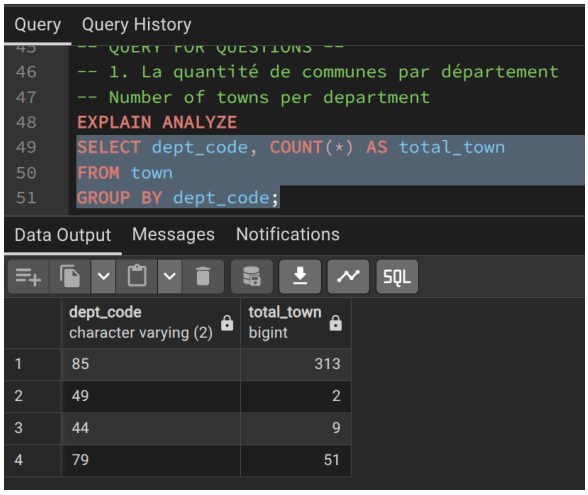
Here are the SQL queries designed to answer specific genealogical questions based on our database schema, along with their results.

1. The number of communes per department

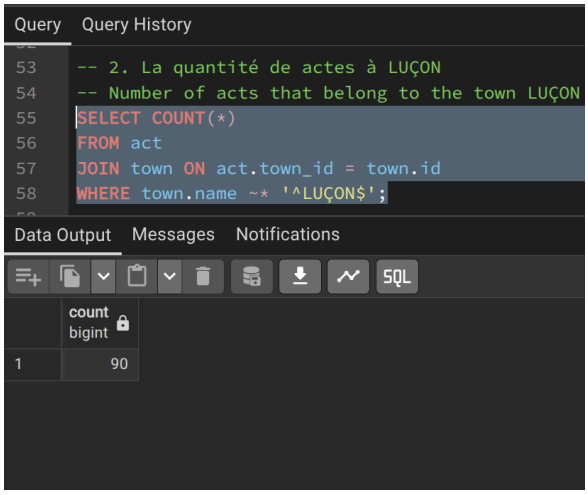
```
SELECT dept_code, COUNT(*) AS total_town
FROM town
GROUP BY dept_code;
```

2. The number of acts in Luçon

```
SELECT COUNT(*)
FROM act
JOIN town ON act.town_id = town.id
WHERE town.name ~* '^LUÇON$';
```



(a) Result for Query 1



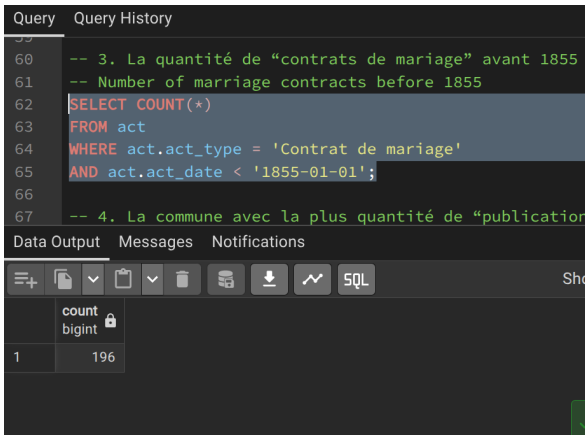
(b) Result for Query 2

3. The number of marriage contracts before 1855

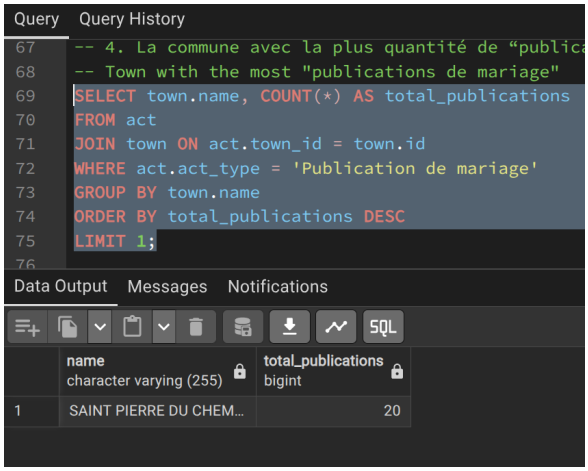
```
SELECT COUNT(*)
FROM act
WHERE act.act_type = 'Contrat de mariage'
AND act.act_date < '1855-01-01';
```

4. The town with the highest number of marriage publications

```
SELECT town.name, COUNT(*) AS total_publications
FROM act
JOIN town ON act.town_id = town.id
WHERE act.act_type = 'Publication de mariage'
GROUP BY town.name
ORDER BY total_publications DESC
LIMIT 1;
```



(a) Result for Query 3



(b) Result for Query 4

5. The date of the first and last records

```
SELECT MIN(act_date) as first_act, MAX(act_date) as last_act
FROM act;
```

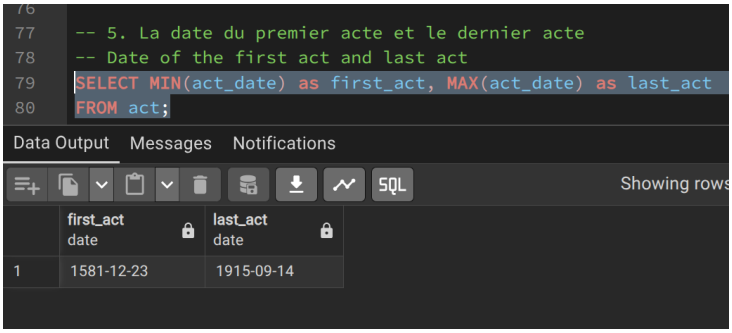


Figure 6: Result for Query 5

Summary of All Query Results

The following image shows the combined results of all queries executed through the Python script.

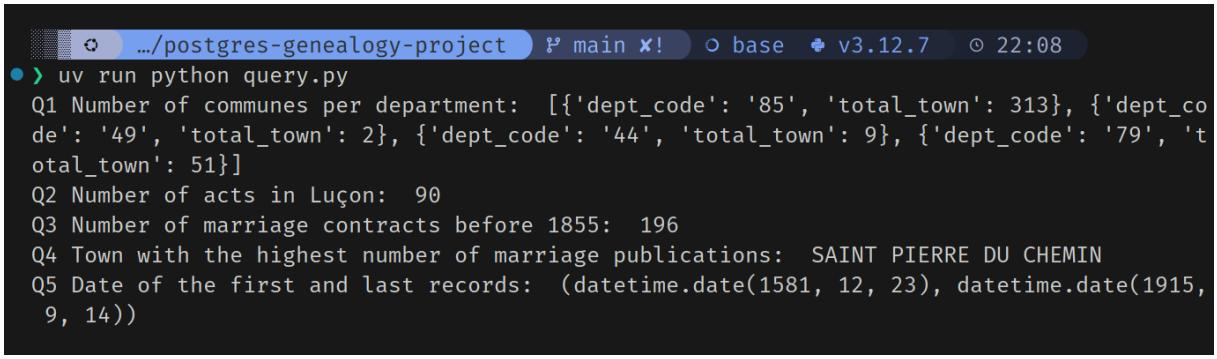


Figure 7: Summary of All Query Results from Python Script

V. Bonus: Scalability & Noisy Data

In this section, we test our database design and ETL pipeline against a much larger and more realistic dataset: a file named `mariages.csv` containing approximately 564,000 records. This file, unlike the initial 5k dataset, contains more noisy and inconsistent data.

V.1. Challenge Overview

The primary goals were:

1. To successfully process and load the 564k records without changing our database schema.
2. To identify and handle the data quality issues ("noise") found in the larger file.
3. To run the same five genealogical queries and compare the results with those from the initial 5k dataset.

V.2. Difficulties Encountered and Solutions

Running our initial script on the new dataset immediately revealed several issues. Here's a summary of the problems we found and how we solved them.

Problem 1: Performance with Large Data

Issue: Reading all 564,000 rows into a pandas DataFrame at once was very slow and used a lot of memory.

Solution: We modified our script to read the CSV in smaller chunks. This meant the program processed the file piece by piece instead of all at once, which significantly improved performance and reduced memory usage.

Problem 2: Inconsistent Date Formats

Issue: The script failed on many rows because the dates were not in the expected `DD/MM/YYYY` format. We found formats that contained only the year of the event like `0́0/00/1731´`, or were missing specific day or month information like `0́0/07/1691´`. These incomplete dates could not be directly parsed.

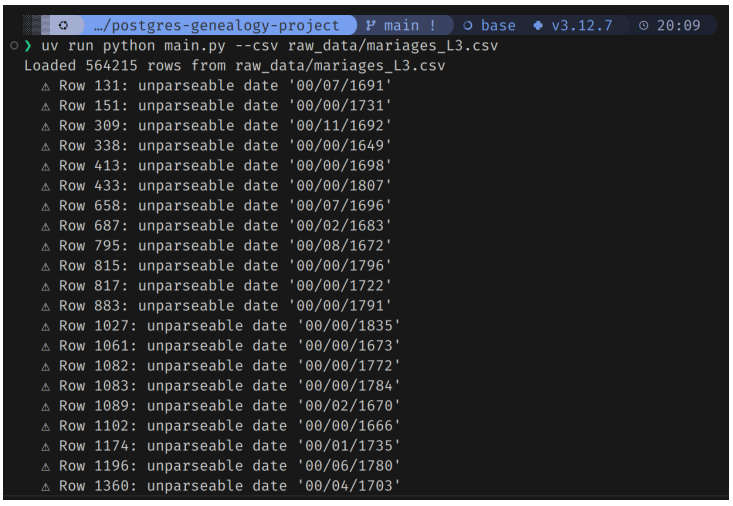


Figure 8: Inconsistent Date Formats

Solution: We improved our date-parsing logic to be more flexible. We used a date-parsing library that can intelligently guess common date formats and added more robust `try-except` blocks to handle and log the formats we couldnt́ automatically convert, rather than crashing the script. For incomplete dates, we decided to store them as 'NULL' in the database to preserve the record while acknowledging the missing information.

Problem 3: Missing Department Codes

Issue: Many rows in the larger dataset had empty or unspecified department codes in the source data. After our initial cleaning, these appeared as `Ńone´` (e.g., `Row 5755: unknown dept_code None - skipped`). Our database schema requires a `town` to be linked to a `department`, so records without a department code could not be processed.

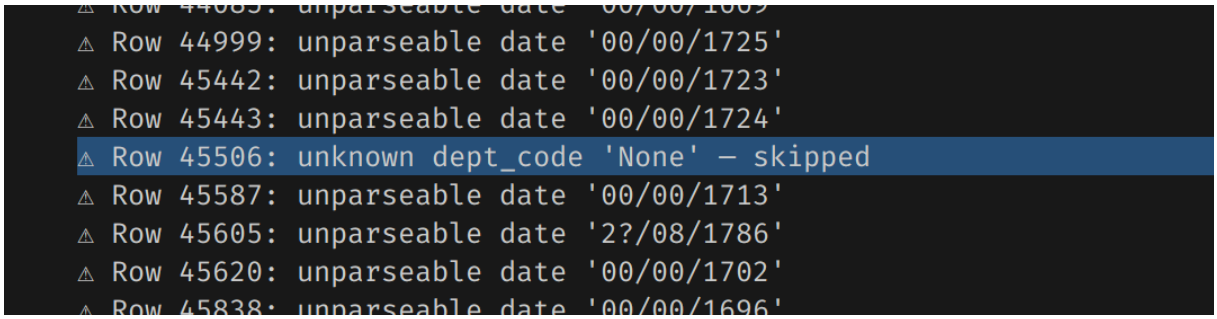


Figure 9: Empty Department Code

Solution: We chose to filter out and log any records where the `dept_code` was missing (i.e., `Ńone´`). This ensures that all data loaded into our `town` table correctly references an existing `department`, maintaining data integrity in accordance with our schema.

V.3. Performance Comparison: 5k vs. 564k Datasets

After cleaning and loading the larger dataset, we re-ran our five queries. The results, shown below, provide a much more comprehensive view of the genealogical data.

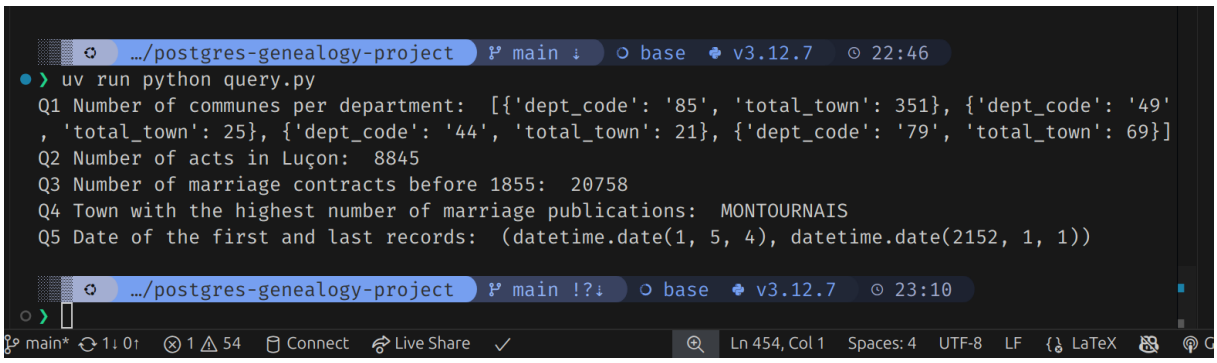


Figure 10: Query results for the 564k dataset

Table 1: Comparison of Query Performance

Query	Time (ms) - 5k Dataset	Time (ms) - 564k Dataset
1. Communes per dept.	0.116	0.057
2. Acts in Luçon	0.834	98.566
3. Contracts before 1855	0.362	47.105
4. Town with most pubs.	0.461	47.565
5. First and last act dates	0.718	94.011

The performance comparison clearly shows that our database design scales effectively. While the 564k dataset is over 100 times larger than the original, the queries run in milliseconds and are still very fast. The most time-consuming queries are those that need to scan the large ‘act‘ table, but even these complete in under 100ms. This demonstrates that our normalized schema and the automatic indexing provided by PostgreSQL are efficient for handling a significantly larger data load.

VI. Conclusion

This project successfully met the goal of designing and implementing a relational database for French genealogical records. Starting from a single, flat CSV file, we designed a normalized schema in Third Normal Form (3NF) to ensure data integrity and minimize redundancy. Our final design consisted of four interconnected tables: `department`, `town`, `person`, and `act`.

A key part of our work was developing a robust ETL (Extract, Transform, Load) pipeline using Python. This pipeline not only cleaned and validated the initial 5,000 records but was also designed to be scalable. As demonstrated in the bonus section, our system is capable of handling a much larger, "noisy" dataset of over 500,000 records with only minor adjustments to handle performance and data inconsistencies.

We proved the effectiveness of our database by running a series of genealogical queries, successfully extracting meaningful information such as marriage statistics and location-based data. Overall, this project provided practical experience in database modeling, data normalization, and building an automated pipeline to handle real-world data challenges.

Appendix: Query Performance Analysis

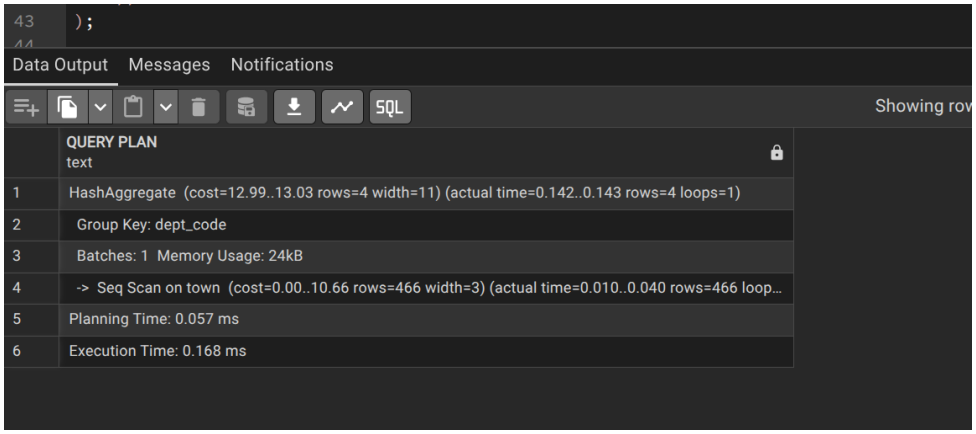


Figure 11: EXPLAIN ANALYZE for Query 1 on the 564k Datasets

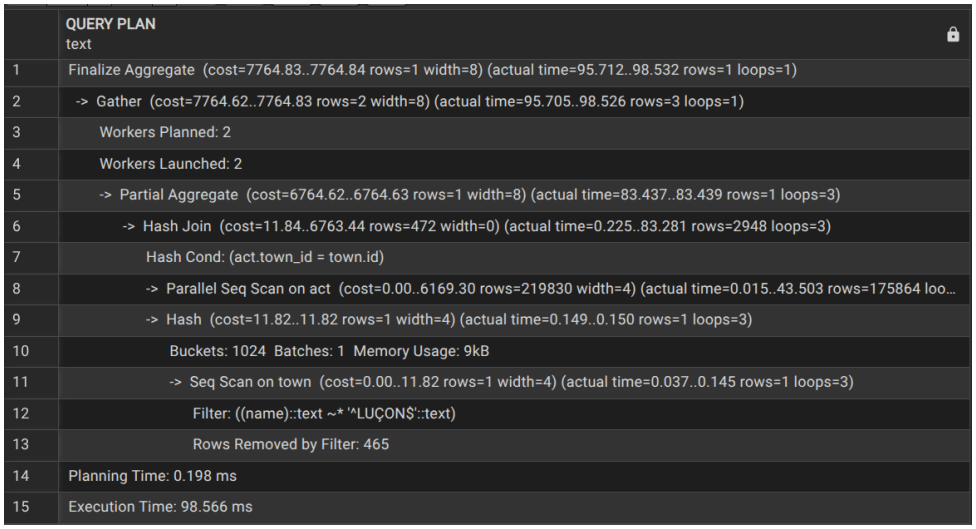


Figure 12: EXPLAIN ANALYZE for Query 2 on the 564k Datasets

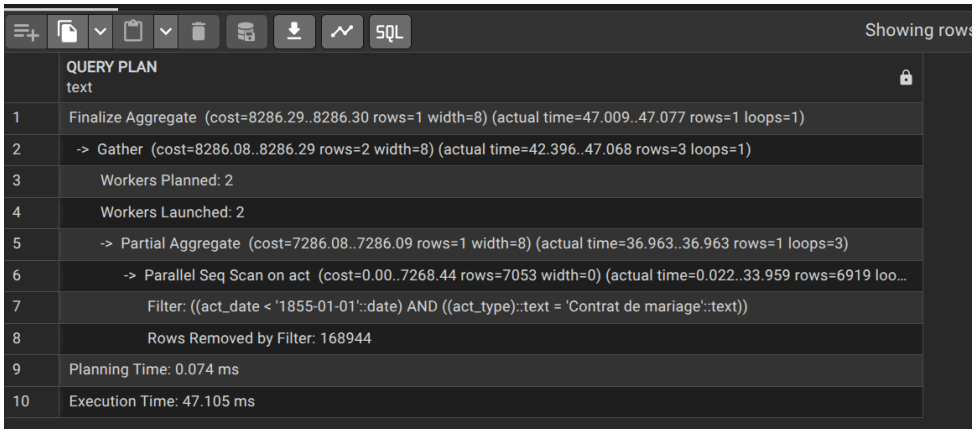


Figure 13: EXPLAIN ANALYZE for Query 3 on the 564k Datasets

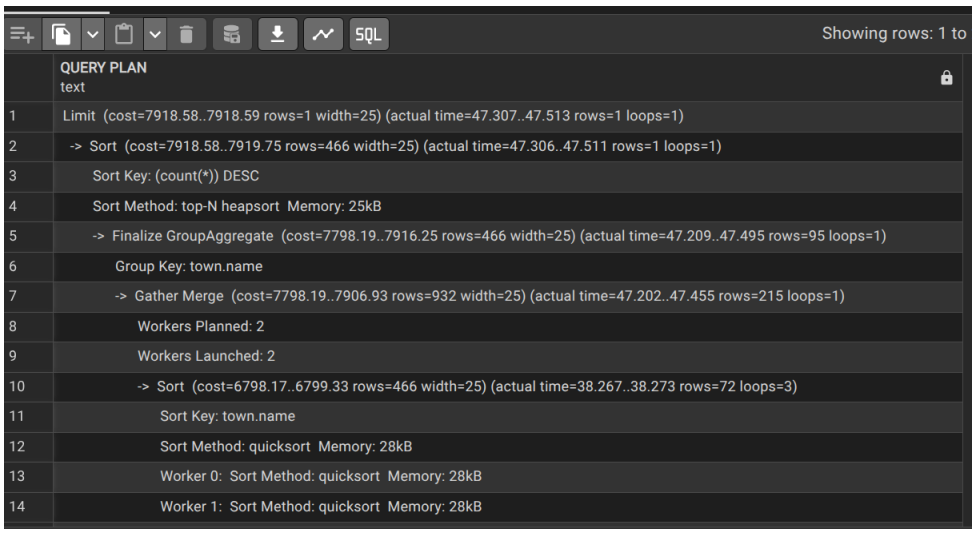


Figure 14: EXPLAIN ANALYZE for Query 4 on the 564k Datasets

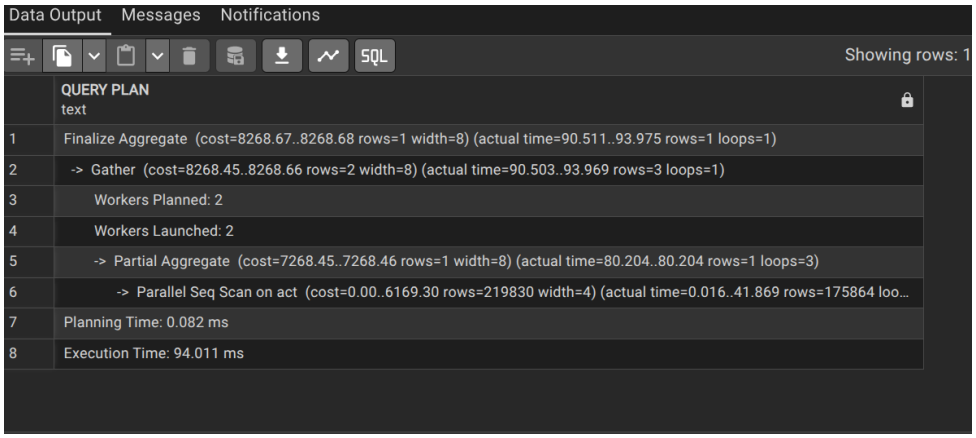


Figure 15: EXPLAIN ANALYZE for Query 5 on the 564k Datasets